



数据库系统

第四章 数据库安全性

胡金鹏



• 问题的提出

- 数据库的一大特点是数据可以共享
- 数据共享必然带来数据库的安全性问题
- 数据库系统中的数据共享不能是无条件的共享

例： 军事秘密、国家机密、新产品实验数据、市场需求分析、市场营销策略、销售计划、客户档案、医疗档案、银行储蓄数据



数据库安全性



数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护



数据库的不完全因素

- 非授权用户对数据库的恶意存取和破坏
 - 一些黑客（Hacker）和犯罪分子在用户存取数据库时猎取用户名和用户口令，然后假冒合法用户偷取、修改甚至破坏用户数据。
 - 数据库管理系统提供的安全措施主要包括**用户身份鉴别**、**存取控制**和**视图**等技术。

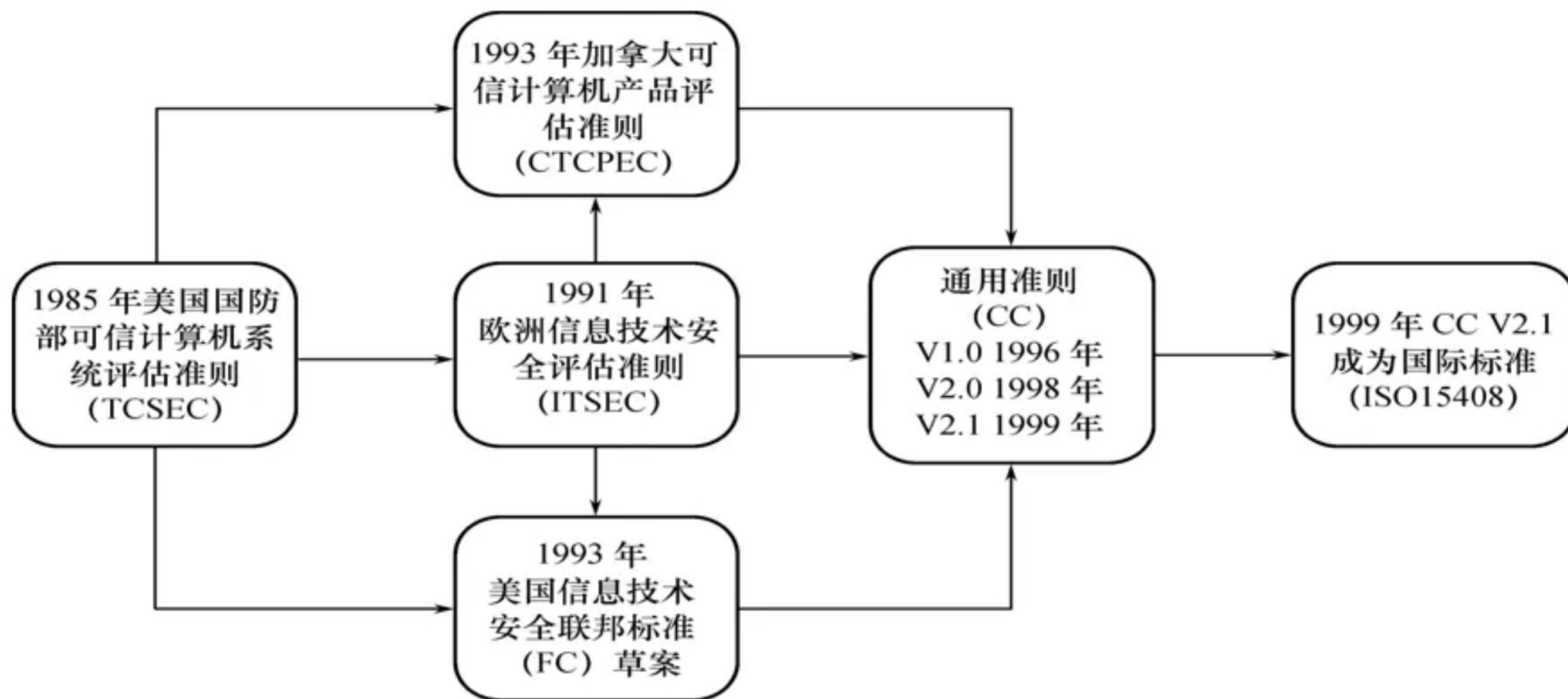
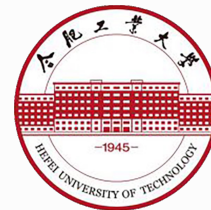


数据库的不完全因素

- **数据库中重要或敏感的数据被泄露**
 - **黑客和敌对分子千方百计盗窃数据库中的重要数据，一些机密信息被暴露。**
 - **数据库管理系统提供的主要技术有强制存取控制、数据加密存储和加密传输等。**
 - **审计日志分析**



数据库安全性





■ 为降低进而消除对系统的安全攻击，各国引用或制定了一系列安全标准

✓ **TCSEC** (Trust Computer System Evaluation Criteria)

(桔皮书)

✓ **TDI** (**Trusted Database Interpretation**) (紫皮书)

■ **TCSEC** (桔皮书)

1985年美国国防部（DoD）正式颁布《DoD可信计算机系统评估标准》（简称**TCSEC**或**DoD85**）。提供一种标准，使用户可以对其计算机系统内敏感信息安全操作的**可信程度**做评估。



■ 为降低进而消除对系统的安全攻击，各国引用或制定了一系列安全标准

✓ **TCSEC** (Trust Computer System Evaluation Criteria)

(桔皮书)

✓ **TDI** (**Trusted Database Interpretation**) (紫皮书)

✓ **TDI** (紫皮书)

1991年4月美国**NCSC**（国家计算机安全中心）颁布了《可信计算机系统评估标准关于可信数据库系统的解释》（**Trusted Database Interpretation** 简称**TDI**。TDI又称紫皮书。它将**TCSEC**扩展到数据库管理系统。TDI中定义了数据库管理系统的设计与实现中需满足和用以进行安全性级别评估的标准。



■ TDI/TCSEC标准的基本内容

TDI与TCSEC一样，从四个方面来描述安全性

级别划分的指标

- 安全策略
- 责任
- 保证
- 文档

安全级别	定义
A1	验证设计 (Verified Design)
B3	安全域 (Security Domains)
B2	结构化保护 (Structural Protection)
B1	标记安全保护 (Labeled Security Protection)
C2	受控的存取保护 (Controlled Access Protection)
C1	自主安全保护 (Discretionary Security Protection)
D	最小保护 (Minimal Protection)

系统可靠或可信程度逐渐增高



■ 四组(**division**)七个等级

D

C (C1, C2)

B (B1, B2, B3)

A (A1)

按系统可靠或可信程度逐渐增高

各安全级别之间具有一种偏序向下兼容的关系，即较高安全性级别提供的安全保护要包含较低级别的所有保护要求，同时提供更多或更完善的保护能力。



■ D级

➤ 将一切不符合更高标准的系统均归于D组

典型例子：DOS是安全标准为D的操作系统
DOS在安全性方面几乎没有什么专门的机制来保障。

■ C1级

- 非常初级的自主安全保护
- 能够实现对用户和数据的分离，进行自主存取控制（DAC），保护或限制用户权限的传播。
- 现有的商业系统稍作改进即可满足

■ C2级

- 是安全产品的最低档次
- 提供受控的存取保护，将C1级的DAC进一步细化，以个人身份注册负责，并实施审计和资源隔离
- 达到C2级的产品在其名称中往往不突出“安全”（Security）这一特色
- 典型例子

Windows 2000 , Oracle 11g



■ B1级

- 标记安全保护。“安全”（ Security ）或“可信的”（ Trusted ）产品。
- 对系统的数据加以标记，对标记的主体和客体实施强制存取控制（ MAC ）、审计等安全机制
- B1级典型例子
 - ✓ 操作系统
 - ✓ 惠普公司的HP-UX BLS release 9.09+
 - ✓ 数据库
 - ✓ Oracle公司的Trusted Oracle
 - ✓ Sybase公司的Secure SQL Server version 11.0.6



■ B2级

- 结构化保护
- 建立形式化的安全策略模型并对系统内的所有主体和客体实施

DAC和MAC

■ B3级

- 安全域
- 该级的可信计算基 (trusted computing base) TCB必须满足访问监控器的要求，审计跟踪能力更强，并提供系统恢复过程

■ A1级

- 验证设计，即提供B3级保护的同时给出系统的形式化设计说明和验证以确信各安全保护真正实现。



数据库安全性

■ CC标准

- 1993年，多个国家开发IT安全通用准则（common criteria，CC），经过多次讨论和修改后，CC2.1版被采用为国际标准。
- 把信息产品的安全要求分为
 - ✓ 安全功能要求
 - ✓ 安全保证要求

■ CC（EAL）划分

- 评估保证级，从EAL1到EAL7共分为7级。

评估保证级	定 义	TCSEC安全级别 (近似相当)
EAL1	功能测试	
EAL2	结构测试	C1
EAL3	系统地测试和检查	C2
EAL4	系统地设计、测试和复查	B1
EAL5	半形式化设计和测试	B2
EAL6	半形式化验证的设计和测试	B3
EAL7	形式化验证的设计和测试 (formally verified design and tested)	A1

保证程度逐渐提高



数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护



非法使用数据库的情况包括：

- 用户编写一段合法的程序绕过DBMS及其授权机制，通过操作系统直接存取、修改或备份数据库中的数据
- 直接或编写应用程序执行非授权操作
- 通过多次合法查询数据库从中推导出一些保密数据

例如：某数据库应用系统禁止查询个人的工资，但允许查任意一组人的平均工资。用户甲想了解张三的工资，于是他：

- 1、首先查询包括张三在内的一组人的平均工资
- 2、然后查用自己替换张三后这组人的平均工资
- 3、从而推导出张三的工资

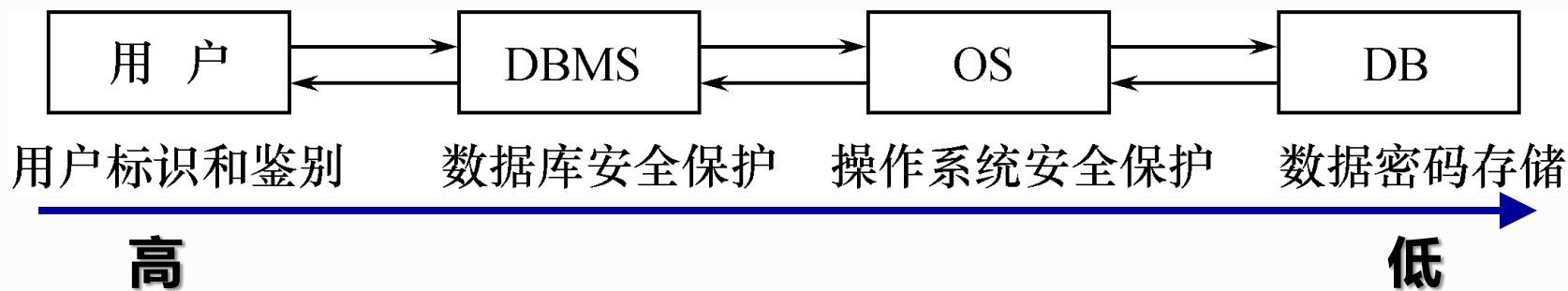
■ 破坏安全性的行为可能是无意的、故意的，或恶意的。



数据库安全性



- 在计算机系统中，安全措施是一级一级层层设置的



- 数据库的安全性可以划分为

三个层次：

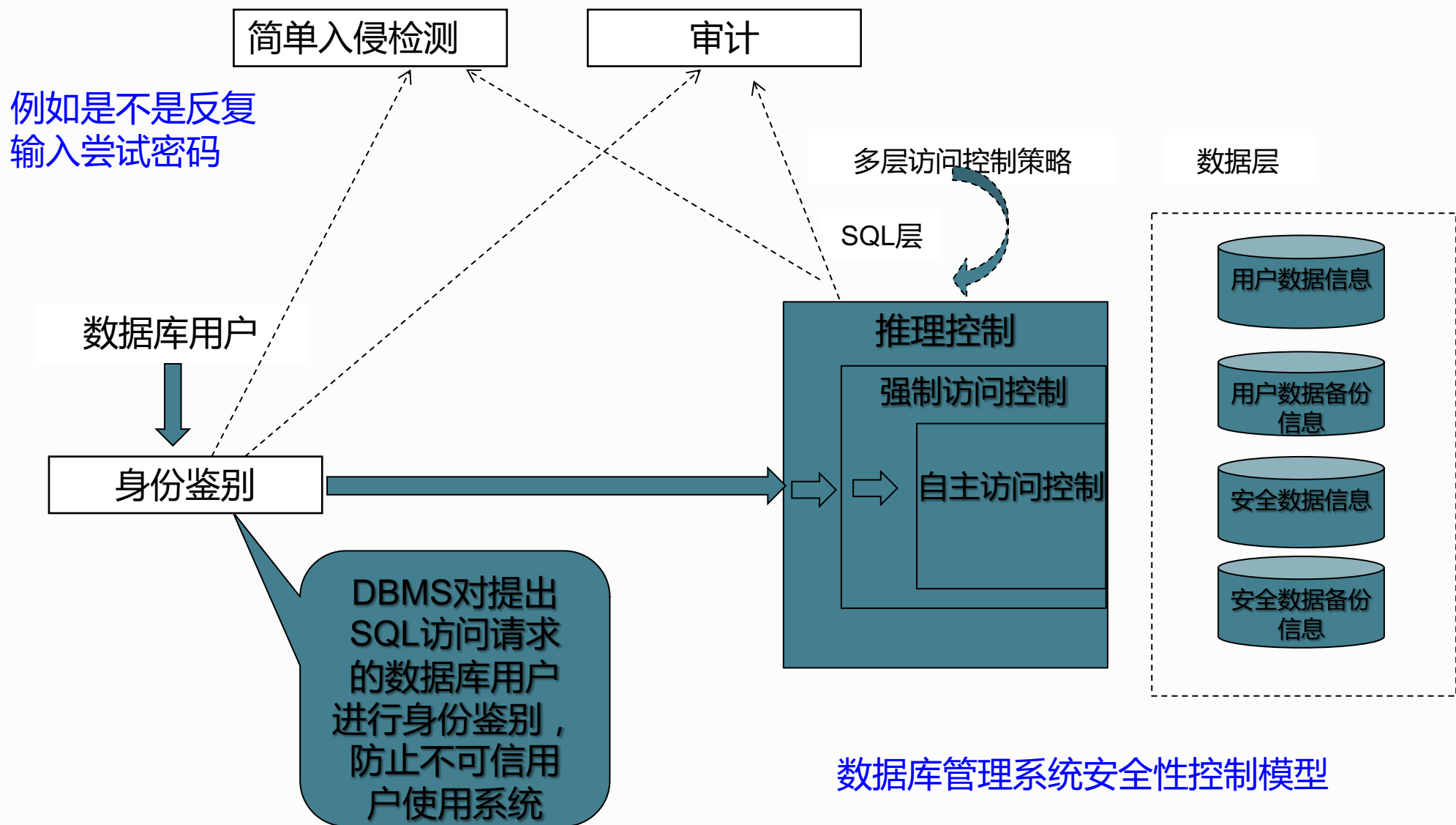
- ✓ 网络系统的安全
- ✓ 操作系统的安全
- ✓ 数据库管理系统的安全

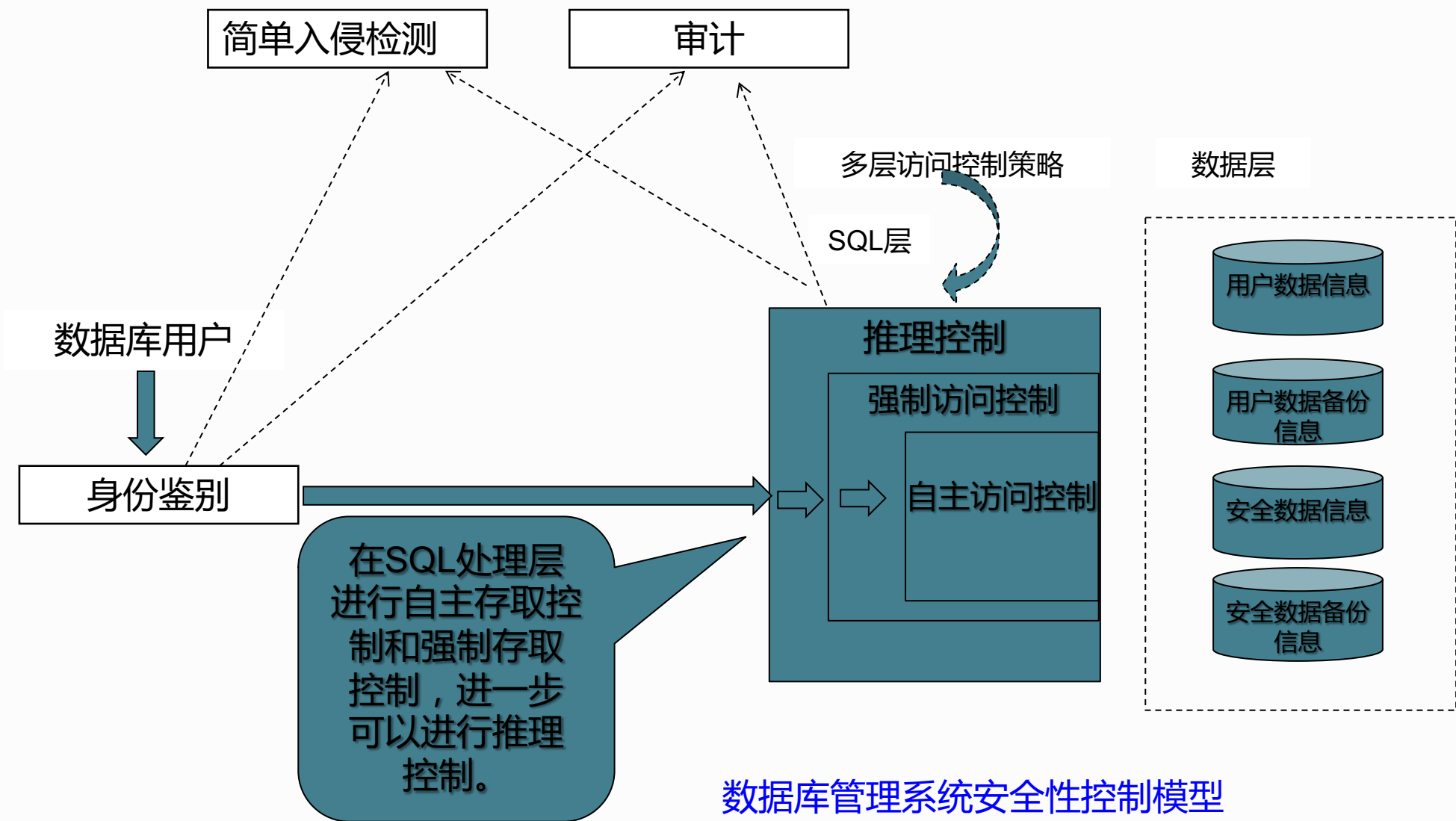
- 数据库安全性控制的常用方法包括：

- 用户标识和鉴别
- 存取控制：只允许用户进行合法操作
- 视图
- 审计：用户退出系统，需进行安全审计。
- 加密存储 等等



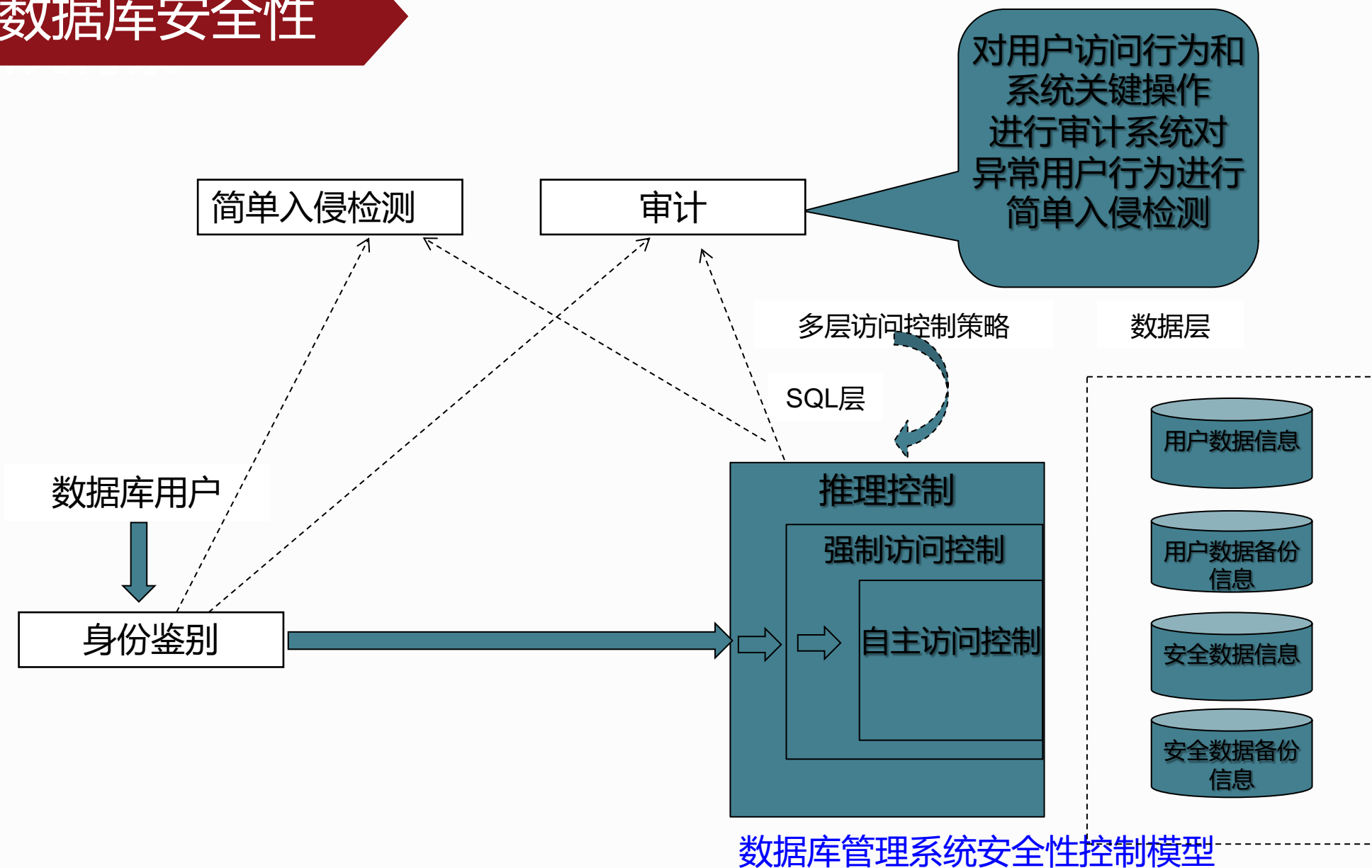
数据库安全性







数据库安全性





- **数据库安全性控制的常用方法**
 - **用户标识和鉴定**
 - **存取控制**
 - **视图**
 - **审计**
 - **加密存储**



用户标识与鉴别 (Identification & Authentication)

系统提供的最外层安全保护措施。

基本方法是：

- 系统提供一定的方式让用户标识自己的名字或身份；
- 系统内部记录着所有合法用户的标识；
- 每次用户要求进入系统时，由系统核对用户提供的身份标识；
- 通过鉴定后才提供机器使用权；
- 用户标识和鉴定可以重复多次。



1. 用户标识 (User Identification)

用一个用户名 (User Name) 或用户标识号 (UID) 来标明用户身份。

2. 口令 (Password)

系统核对口令以鉴别用户身份。

通过用户名和口令的方法简单易行，但这些信息容易被人窃取。可以采取更复杂的方法——**每个用户预先约定好一个计算过程或者函数。** (动态口令)

- 系统提供一个随机数
- 用户根据自己预先约定的计算过程或者函数进行计算
- 系统根据用户计算结果是否正确鉴定用户身份

3. 生物特征鉴别

4. 智能卡鉴别



静态口令

这种方式是当前常用的鉴别方法。

静态口令一般由用户自己设定，鉴别时只要按要求输入正确的口令，系统将允许用户使用数据库管理系统。

这些口令是静态不变的，在实际应用中，用户常常用自己的生日、电话、简单易记的数字等内容作为口令，很容易被破解。而一旦被破解，非法用户就可以冒充该用户使用数据库。

这种方式虽然简单，但容易被攻击，安全性较低。



动态口令

它是目前较为安全的鉴别方式。

这种方式的口令是动态变化的，每次鉴别时均需使用动态产生的新口令登录数据库管理系统，即采用一次一密的方法。

常用的方式如短信密码和动态令牌方式，每次鉴别时要求用户使用通过短信或令牌等途径获取的新口令登录数据库管理系统。

与静态口令鉴别相比，这种认证方式增加了口令被窃取或破解的难度，安全性相对高一些。



生物特征鉴别

它是一种通过生物特征进行认证的技术，其中，生物特征是指生物体唯一具有的，可测量、识别和验证的稳定生物特征，如指纹、虹膜和掌纹等。

这种方式通过采用图像处理和模式识别等技术实现了基于生物特征的认证，- 与传统的口令鉴别相比，无疑产生了质的飞跃，安全性较高。

智能卡鉴别

智能卡是一种不可复制的硬件，内置集成电路的芯片，具有硬件加密功能。

智能卡由用户随身携带，登录数据库管理系统时用户将智能卡插入专用的读卡器进行身份验证。



数据库安全性



- 数据库安全性所关心的主要是DBMS的**存取控制**机制。
- 数据库安全最重要的一点就是确保只授权给有资格的用户访问数据库的权限，同时令所有未授权的人员无法接近数据，这主要通过数据库系统的存取控制机制实现。
- 存取控制机制主要包括两部分：
 - **定义用户权限**
 - **合法权限检查**
- 用户权限定义和合法权检查机制一起组成了DBMS的安全子系统



■ 定义用户权限，并将用户权限登记到数据字典中

- 用户对某一数据对象的操作权力称为权限。DBMS系统提供适当的语言来定义用户权限，这些定义经过编译后存放在数据字典中，被称做安全规则或授权规则。

■ 合法权限检查

- 每当用户发出存取数据库的操作请求后，DBMS查找数据字典，根据安全规则进行合法权限检查，若用户的操作请求超出了定义的权限，系统将拒绝执行此操作。



■ 常用存取控制方法有：

- **自主存取控制** (Discretionary Access Control , 简称DAC) , **C2级, 灵活**
 - ★ 同一用户对于不同的数据对象有不同的存取权限
 - ★ 不同的用户对同一对象也有不同的权限
 - ★ 用户还可将其拥有的存取权限转授给其他用户
- **强制存取控制** (Mandatory Access Control , 简称 MAC) , **B1级、严格**
 - ★ 每一个数据对象被标以一定的密级
 - ★ 每一个用户也被授予某一个级别的许可证
 - ★ 对于任意一个对象，只有具有合法许可证的用户才可以存取



自主存取控制

	表1	表2
小李	RW	RW
小王	-	R

- 主体小李可以对客体表1和客体表2进行“读 (R)”和“写 (W)”操作
- 主体小王只能对客体表2进行“读 (R)”操作。

强制存取控制

- 人事表Person (机密, {人事, 财务})。
- 人事部门主管小王 (绝密, {人事, 财务})。
- 小王的标签 (绝密, {人事, 财务}) 中敏感级别比数据表Person的敏感级别高 (机密, {人事, 财务})，那么小王就可以读数据表Person中的数据。
- 客体资源配置表
 - 资源: 人事表Person
 - 主体: 财务人员，人事人员
 - 等级: 机密级
 - 操作: 查看主体配置表
- 主体: 小王 类别: 人事, 财务 等级: 绝密级



一、自主存取控制

- 通过 SQL 的 **GRANT** 语句和 **REVOKE** 语句实现
- 用户权限组成
 - 数据对象
 - 操作类型
- 定义用户存取权限：定义用户可以在哪些数据库对象上进行哪些类型的操作
- 定义存取权限称为**授权**



一、自主存取控制

关系数据库系统中存取控制对象

对象类型	对象	操作类型
数据库	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
模式	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
数据	属性列	SELECT, INSERT, UPDATE, REFERENCES ALL PRIVILEGES



一、自主存取控制

授权与回收：授权语句

GRANT语句的一般格式：

GRANT <权限> [, <权限>]...

[ON <对象类型> <对象名>]

TO <用户> [, <用户>]...

[WITH GRANT OPTION];

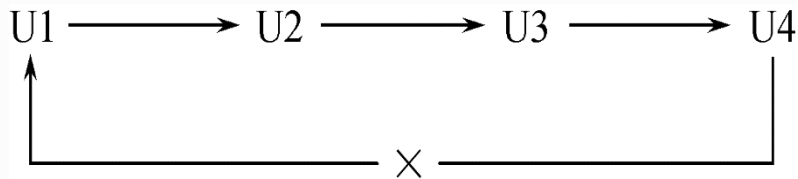
语义：将对指定操作对象的指定操作权限授予指定的用户



一、自主存取控制

【说明】

- 发出GRANT可以是数据库管理员、数据库对象创建者、拥有该权限的用户
- 接受权限的用户可以是一个用户、多个用户、PUBLIC（即全体用户）
- WITH GRANT OPTION语句：有，表示该权限可以再授予其他用户，没有，表示该权限不可以再授予其他用户。
- 不允许循环授权





一、自主存取控制

【例1】把查询Student表权限授给用户U1

```
GRANT SELECT  
ON TABLE Student  
TO U1;
```

【例2】把对Student表和Course表的全部权限授予用户U2和U3

```
GRANT ALL PRIVILIGES  
ON TABLE Student, Course  
TO U2, U3;
```



一、自主存取控制

【例3】把对表SC的查询权限授予所有用户

```
GRANT SELECT  
ON TABLE SC  
TO PUBLIC;
```

【例4】把查询Student表和修改学生学号的权限授给用户U4

```
GRANT UPDATE(Sno), SELECT  
ON TABLE Student  
TO U4;
```

对属性列的授权时必须明确指出相应属性列名



一、自主存取控制

【例5】把对表SC的INSERT权限授予U5用户，并允许他再将此权限授予其他用户

```
GRANT INSERT  
ON TABLE SC  
TO U5
```

```
WITH GRANT OPTION;
```

执行后，U5不仅拥有了对表SC的INSERT权限，还可以传播此权限



一、自主存取控制

**【例6】 GRANT INSERT ON TABLE SC TO U6
WITH GRANT OPTION;**

(可以由U5运行该指令)

同样，U6还可以将此权限授予U7：

**【例7】 GRANT INSERT
ON TABLE SC
TO U7;**

但U7不能再传播此权限。



下表是执行了 [例1] 到 [例7] 的语句后，学生-课程数据库中的用户权限定义表

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系Student	SELECT	不能
DBA	U2	关系Student	ALL	不能
DBA	U2	关系Course	ALL	不能
DBA	U3	关系Student	ALL	不能
DBA	U3	关系Course	ALL	不能
DBA	PUBLIC	关系SC	SELECT	不能
DBA	U4	关系Student	SELECT	不能
DBA	U4	属性列Student.Sno	UPDATE	不能
DBA	U5	关系SC	INSERT	能
U5	U6	关系SC	INSERT	能
U6	U7	关系SC	INSERT	不能



一、自主存取控制

授权与回收：回收语句

REVOKE语句的一般格式为：

REVOKE <权限> [, <权限>]...

[ON <对象类型> <对象名>]

FROM <用户> [, <用户>]...;

语义：授予的权限由DBA或其他授权者用REVOKE语句收回



一、自主存取控制

【例8】把用户U4修改学生学号的权限收回

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

【例9】收回所有用户对表SC的查询权限

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```



一、自主存取控制

【例10】把用户U5对SC表的INSERT权限收回

```
REVOKE INSERT  
ON TABLE SC  
FROM U5 CASCADE ;
```

- 将用户U5的INSERT权限收回的时候必须级联（ CASCADE ）收回
- 系统只收回直接或间接从U5处获得的权限



执行 [例8] 到 [例10] 的语句后，学生-课程数据库中的用户权限定义表

授权用户名	被授权用户名	数据库对象名	允许的操作类型	能否转授权
DBA	U1	关系Student	SELECT	不能
DBA	U2	关系Student	ALL	不能
DBA	U2	关系Course	ALL	不能
DBA	U3	关系Student	ALL	不能
DBA	U3	关系Course	ALL	不能
DBA	U4	关系Student	SELECT	不能



【总结】

- **数据库管理员的授权：**

拥有所有对象的所有权限

根据实际情况不同的权限授予不同的用户

- **用户的授权：**

拥有自己建立的对象的全部的操作权限

可以使用grant把权限授予其他用户。

- **被授予用户的权限**

如果有“继续授权”许可，可以把获得的权限再授予其他用户。

所有授予出去的权限必要时又可以用revoke语句收回



数据库安全性

■ 创建数据库模式的权限

- DBA在创建用户时实现！

CREATE USER <username>
[WITH] [DBA | RESOURCE | CONNECT]

不是SQL标准

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库 执行数据查询 和操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	可以	可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

注：CREATE USER不是SQL标准，各个系统的实现相差甚远

数据库角色

■基于角色的访问控制RBAC(Role-Based Access Control)

- ✓数据库角色：被命名的一组与数据库操作相关的权限，角色是权限的集合。
- ✓角色是权限的集合。
- ✓可以为一组具有相同权限的用户创建一个角色。
- ✓简化授权的过程。
- ✓每个角色名称必须是唯一的

■角色的创建：**CREATE ROLE** <角色名> ;

■给角色授权：

GRANT <权限> [, <权限>] ... **ON** <对象类型> 对象名
TO <角色> [, <角色>] ... ;



数据库角色

■ 将一个角色授予其他的角色或用户：

GRANT <角色1> [, <角色2>] ... **TO** <角色3> [, <用户1>]

...

[**WITH ADMIN OPTION**] ;

■ 角色权限的收回：

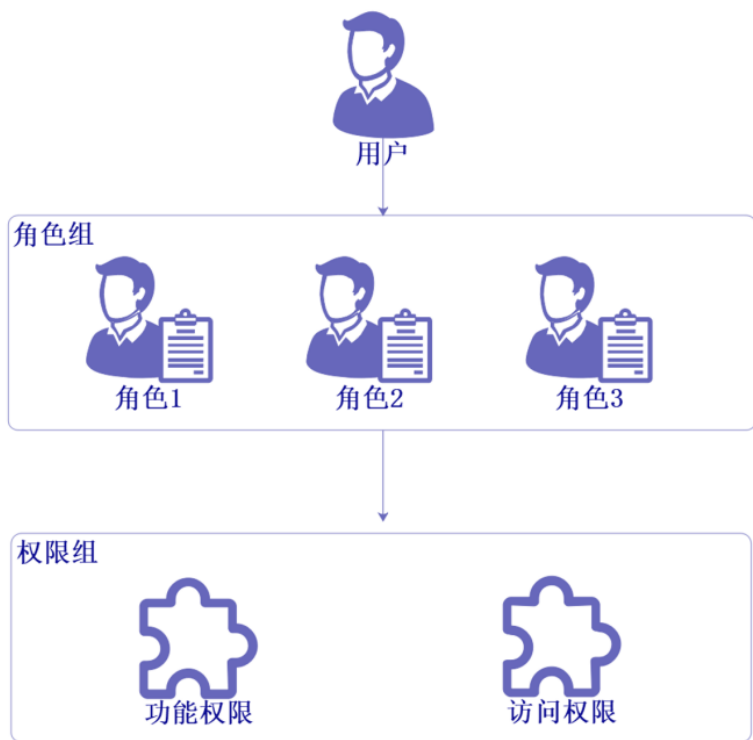
REVOKE <权限> [, <权限>] ... **ON** <对象类型> <对象名>

FROM <角色> [, <角色>] ... ;



数据库角色

■基于角色的访问控制RBAC(Role-Based Access Control)



□ 权限表

- 名称: 创建客户 资源: 客户信息 操作: 创建
- 名称: 删除客户 资源: 客户信息 操作: 删除
- 名称: 查看客户 资源: 客户信息 操作: 查看
- 名称: 修改客户 资源: 客户信息 操作: 修改

□ 角色表

- 名称: 销售角色
 - 权限: 创建客户、删除客户、查看客户、修改客户

□ 用户表

- 主体: 小王 角色: 销售角色

数据库角色

■ 将一个角色授予其他的角色或用户：

GRANT <角色1> [, <角色2>] ... **TO** <角色3> [, <用户1>]

...

[WITH ADMIN OPTION] ;

- 当为用户授予角色时，相当于为用户授予了多种权限。
- 避免了向用户逐一授权，从而简化了用户权限的管理。



数据库角色

■ 角色权限的收回 :

```
REVOKE <权限> [ , <权限> ] ... ON <对象类型> <对象名>  
FROM <角色> [ , <角色> ] ... ;
```

数据库角色

【例11】通过角色来实现将一组权限授予与收回给用户。

步骤如下：

1. 首先创建一个角色 R1

```
CREATE ROLE R1 ;
```

2. 然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限

```
GRANT SELECT , UPDATE , INSERT  
ON TABLE Student  
TO R1 ;
```



数据库角色

【例11】通过角色来实现将一组权限授予与收回给用户。

步骤如下：

3. 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限

GRANT R1

TO 王平，张明，赵玲；

4. 可以一次性通过R1来回收王平的这3个权限

REVOKE R1

FROM 王平；



自主存取控制的缺点

可能存在数据的“无意泄露”的风险。

原因：这种机制仅仅通过对数据的存取权限来进行安全控制，而数据本身并无安全性标记

解决：对系统控制下的所有主客体实施强制存取控制策略



二、强制存取控制（MAC）

- MAC是系统为保证更高层次的安全性，按照TDI/TCSEC标准中安全策略的要求，所采取的强制存取检查手段。
- MAC不是用户能直接感知或进行控制的。
- MAC适用于对数据有严格而固定密级分类的部门，例如军事部门或政府部门。



二、强制存取控制（MAC）

在MAC中，DBMS所管理的全部实体被分为两大类：

1. **主体** —— 系统中的活动实体，包括：
 - DBMS所管理的实际用户
 - 代表用户的各进程
2. **客体** —— 系统中的被动实体，是受主体操纵的，包括：
 - 文件
 - 基表
 - 索引
 - 视图 等



二、强制存取控制 (MAC)

对于主体和客体，DBMS为它们的每个实例（值）指派一个**敏感度标记（Label）**。敏感度标记被分为若干级别，例如：

- ★ **绝密（Top Secret , TS）**
- ★ **机密（Secret , S）**
- ★ **可信（Confidential,C）**
- ★ **公开（Public,P）等**

- 主体的敏感度标记称为**许可证级别（Clearance Level）**；
- 客体的敏感度标记称为**密级（Classification Level）**。
- MAC机制就是通过对比主体的Label和客体的Label，最终确定主体是否能够存取客体。



二、强制存取控制 (MAC)

强制存取控制规则：

- 当某一用户（或某一主体）以标记Label注册入系统时，系统要求他对任何客体的存取必须遵循下面两条规则：
 1. 仅当主体的许可证级别**大于或等于**客体的密级时，该主体才能**读取**相应的客体。
 2. 仅当主体的许可证级别**等于**客体的密级时，该主体才能**写**相应的客体。

即：

- 用户S, 不能读取数据对象O, 除非 $\text{Level}(S) \geq \text{Level}(O)$
- 用户S, 不能写数据对象O, 除非 $\text{Level}(S) = \text{Level}(O)$ 。



二、强制存取控制 (MAC)

某些系统将第 (2) 条修正为：

2. 仅当主体的许可证级别**小于或等于**客体的密级时，主体才能写客体。

即：用户S, 不能写数据对象O, 除非 $\text{Level}(S) \leq \text{Level}(O)$ 。

- 也就是说，用户可为写入的数据对象赋予高于自己的许可证级别的密级
- 这样，一旦数据被写入，该用户自己也不能再读该数据对象了。
- 这两种规则的共同点是：禁止了拥有高许可证级别的主体更新低密级的数据对象，从而防止了敏感数据的泄露。



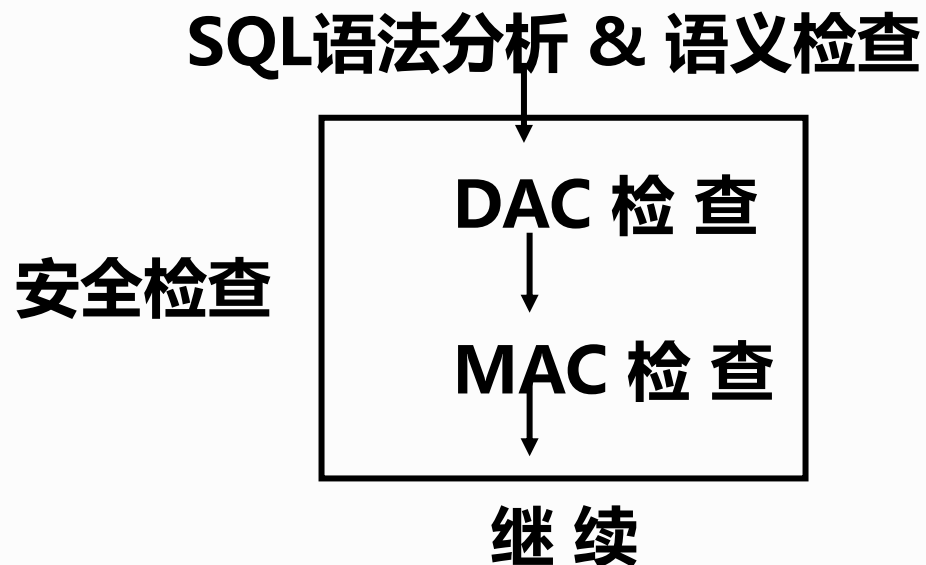
MAC是对数据本身进行密级标记，无论数据如何复制，标记与数据不可分的整体，只有符合密级的用户才能操纵数据。

DAC与MAC共同构成DBMS的安全机制

实现MAC时要首先实现DAC

原因：较高安全性级别提供的安全保护要包含较低级别的所有保护

DAC + MAC安全检查示意图



先进行DAC检查，通过DAC检查的数据对象再由系统进行MAC检查，只有通过MAC检查的数据对象方可存取。



数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护

视图机制

- 可以通过**视图机制**把要保密的数据对无权存取这些数据的用户**隐藏**起来，对数据提供一定程度的安全保护。
- **可以将视图机制与授权机制配合使用：**
 - 首先用视图机制屏蔽掉一部分保密数据
 - 视图上面再进一步定义存取权限
 - 间接实现了支持存取谓词的用户权限定义
- **视图机制的主要功能在于提供数据独立性**，其安全保护功能不太精细，往往远不能达到应用系统的要求。



视图机制

【例12】建立计算机系学生的视图，把对该视图的SELECT权限授予王平，把该视图上的所有操作权限授予张明

1、先建立计算机系学生的视图CS_Student

```
CREATE VIEW CS_Student  
AS  
SELECT *  
FROM Student  
WHERE Sdept='CS' ;
```



视图机制

【例12】建立计算机系学生的视图，把对该视图的SELECT权限授于王平，把该视图上的所有操作权限授于张明

2、在视图上进一步定义存取权限

```
GRANT SELECT  
ON CS_Student  
TO 王平 ;
```

```
GRANT ALL PRIVILIGES  
ON CS_Student  
TO 张明 ;
```




数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护

审计

什么是审计？

- 启用一个专用的**审计日志 (Audit Log)**，将用户对数据库的所有操作记录在上面
- DBA可以利用审计日志中的追踪信息，找出非法存取数据的人、时间和内容。（例如对失败的登录次数进行审计，某个数据库上的DDL语句进行审计，某个数据库表里面的delete语句进行审计）

C2以上安全级别的DBMS必须具有审计功能

DBMS往往将其作为可选特征

- 审计很费时间和空间
- DBA可以根据应用对安全性的要求，灵活地打开或关闭审计功能。
- 审计功能一般主要用于安全性要求较高的部门

强制性机制：用户识别和鉴定、存取控制、视图

预防监测手段： 审计技术

审计 审计事件

服务器事件 审计数据库服务器发生的事件。（服务器登录等）

系统权限事件 对系统拥有的结构或模式对象进行操作的审计，要求该操作的权限是通过系统权限获得的。
（例如建立模式）

语句事件 对SQL语句，如DDL、DML、DQL及DCL语句的审计。

DDL（数据定义语言） DML（数据操作语言） DQL（数据查询语言） DCL(数据控制语言)

模式对象事件 对特定模式对象上进行的SELECT或DML操作的审计。



审计

审计功能

1. 基本功能：提供多种审计查阅方式提供多种审计查阅方式。
2. 多套审计规则：一般在初始化设定。
3. 提供审计分析和报表功能。
4. 审计日志管理功能：①防止审计员误删审计记录，审计日志必须先转储后删除；②对转储的审计记录文件提供完整性和保密性保护；③只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等；
5. 提供查询审计设置及审计记录信息的专门视图。



审计

审计一般可以分为：

1. 用户级审计

任何用户都可设置的审计，主要**针对用户自己创建的数据库表或视图进行审计**，记录所有用户对**这些表或视图的一切成功和（或）不成功的访问要求以及各种类型的SQL操作**。

2. 系统级审计

只能由DBA设置，用以**监测成功或失败的登录要求、监测GRANT和REVOKE操作以及其他数据库级权限下的操作**。



审计 应用场景

为操作进行记录，以便在事后进行追踪和问责

调查可疑活动

监视和收集有关特定数据库活动的数据库

对选定用户的数据库操作进行监视和记录

审计

AUDIT语句：设置审计功能

NOAUDIT语句：取消审计功能

【例13】对修改SC表结构或修改SC表数据的操作进行审计

```
AUDIT ALTER , UPDATE
```

```
ON SC ;
```

【例14】取消对SC表的审计

```
NOAUDIT ALTER , UPDATE
```

```
ON SC
```



审计

审计设置和审计日志一般存储在数据字典中。

必须打开审计开关，系统参数 `audit_trail` 设为true，才能在系统表SYS_AUDITTRAIL中看到审计信息。

数据库安全审计系统提供一种事后检查的安全机制。



数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护

数据加密

防止数据库中数据在存储和传输中失密的有效手段。

- 加密的基本思想

根据一定的算法将原始数据（术语为明文，Plain text）
变换为不可直接识别的格式（术语为密文，Cipher text）。
不知道解密算法的人无法获知数据的内容。

- 数据加密以后，在数据库表中存储的是密文，系统管理员也不能见到明文
提高了关键数据的安全性。



一、存储加密

➤ 透明存储加密

- ✓ 内核级加密保护方式，对用户完全透明
- ✓ 将数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密
- ✓ 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可
- ✓ 内核级加密方法: 性能较好，安全完备性较高

■ 非透明存储加密

数据库用户通过多个加密函数实现



二、传输加密

➤ 链路加密

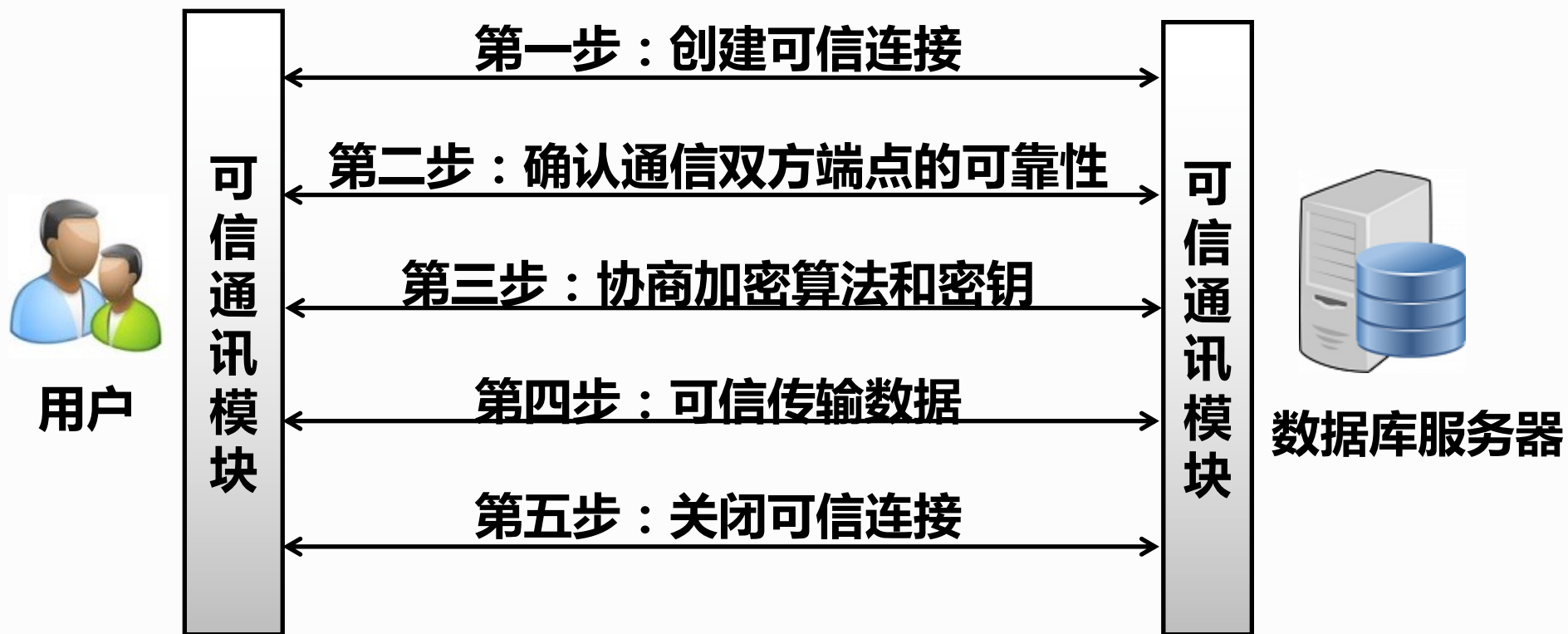
- 在链路层进行加密
- 传输信息由报头和报文两部分组成
- 报文和报头均加密

➤ 端到端加密

- 在发送端加密，接收端解密
- 只加密报文不加密报头
- 所需密码设备数量相对较少，容易被非法监听者发现并从中获取敏感信息



数据库安全性



数据库管理系统可信传输示意图



■ 基于安全套接层协议SSL传输方案的实现思路：

(1) 确认通信双方端点的可靠性

- 采用基于**数字证书 (CA)**的服务器和客户端认证方式
- 通信时均首先向对方提供己方证书，然后使用本地的CA (Certificate Authority),信任列表和证书撤销列表对接收到的对方证书进行验证

(2) 协商加密算法和密钥

- 确认双方端点的可靠性后，通信双方协商本次会话的加密算法与密钥



■ 基于安全套接层协议SSL传输方案的实现思路：

(3) 可信数据传输

- 业务数据在被发送之前将被用某一组特定的密钥进行加密和消息摘要计算，以密文形式在网络上传输
- 当业务数据被接收的时候，需用相同一组特定的密钥进行解密和摘要计算



数据库安全性



4.1 数据库安全性概述

4.2 数据库安全性控制

4.3 视图机制

4.4 审计 (Audit)

4.5 数据加密

4.6 其他安全性保护

推理控制

- 处理强制存取控制未解决的问题
- 避免用户利用能够访问的数据推知更高密级的数据
- 常用方法
 - 基于函数依赖的推理控制
 - 基于敏感关联的推理控制
- 允许用户查询**聚集**类型的信息（例如合计、平均值等）
- 不允许查询**单个**记录信息

例：允许查询“程序员的平均工资是多少？”

不允许查询“程序员张勇的工资？”



• 隐蔽信道

- 处理强制存取控制未解决的问题
- 在实际应用中，攻击者制造一组表面上合法的操作系列，将高级数据库传送到低级用户。这种隐蔽的非法通道叫**隐蔽通道**。这种通信方式往往不被系统的存取控制机制所检测和控制。
- 利用数据库中的共享资源，如数据、数据字典等。发送者修改数据/数据字典，接受者则通过完整性约束等方式间接感知数据/数据字典的修改，以此来传输机密信息。



- **数据隐私保护**

- 描述个人控制其不愿他人知道或他人不便知道的个人数据的能力
- 范围很广：数据收集、数据存储、数据发布和数据发布等各个阶段



【例】下面两个查询都是合法的：

- 1．本公司共有多少女高级程序员？**
- 2．本公司女高级程序员的工资总额是多少？**

如果第一个查询的结果是“1”，

那么第二个查询的结果显然就是这个程序员的工资数。（涉及该女高级程序员的个人隐私）

规则1：任何查询至少要涉及N(N足够大)个以上的记录



【例】 下面两个查询都是合法的：

```
SELECT COUNT(*) FROM employees
```

```
WHERE gender = '女' AND position = '高级程序员' ;
```

```
SELECT SUM(salary) FROM employees
```

```
WHERE gender = '女' AND position = '高级程序员';
```

规则1：

-- 查询1返回1，触发阈值保护

```
SELECT SUM(salary) FROM employees
```

```
WHERE gender = '女' AND position = '高级程序员'
```

```
HAVING COUNT(*) >= 5; -- 不满足条件则返回空
```



【例】 用户A发出下面两个合法查询：

1 . 用户A和其他N个程序员的工资总额是多少？

2 . 用户B和其他N个程序员的工资总额是多少？

若第一个查询的结果是X，第二个查询的结果是Y，

由于用户A知道自己的工资是Z，

那么他可以计算出用户B的工资= $Y - (X - Z)$ 。

原因：两个查询之间有很多重复的数据项

规则2：任意两个查询的相交数据项不能超过M个



【例】用户A发出下面两个合法查询：

查询 1：查询全班数学成绩的平均分。

查询 2：查询去掉某个特定学生后的平均分。

计算两者的差值，可以推算该学生的成绩。



可以证明，在上述两条规定下，如果想获知用户B的工资额
A至少需要进行 $1 + (N-2)/M$ 次查询，N是总数，M是单次排查的数量。

规则3：任一用户的查询次数不能超过 $1 + (N-2)/M$

如果两个用户合作查询就可以使这一规定失效
数据库安全机制的设计目标：

试图破坏安全的人所花费的代价 $\gg$ 得到的利益



- **数据库的安全性**
 - 是指保护数据库，防止因用户非法使用数据库造成数据泄露、更改或破坏。
- **实现数据库系统安全性的技术和方法**
 - **用户标识与鉴别**
 - **存取控制**
 - 自主存取控制方法：**授权和回收、权利传播**
 - 强制存取控制方法：**数据对象分级、用户分级**
 - **视图机制**
 - **审计**
 - **数据加密**